🧠 PROJECT NAME (Proposal)

AegisHire — Autonomous AI HR & Technical Interview Intelligence Platform

1 Executive Vision

AegisHire is a **multi-agent AI HR platform powered by MCP Server**, designed to:

- Automatically evaluate developer candidates
- Generate personalized interview questions from real Git repositories
- Detect AI-assisted cheating during interviews
- Monitor and auto-patch enterprise applications using AI
- Provide strictness levels depending on enterprise plan
- Offer enterprise HR analytics
- Maintain audit transparency

This is not just an interview app.
This is an **AI-powered Technical Hiring Infrastructure**.

2 Core Functional Domains

◇ A. Candidate Intelligence Engine

The system:

- Connects to GitHub / GitLab
- Analyzes repositories
- Extracts:
    - Tech stack
    - Code complexity
    - Architecture style
    - Commit patterns
    - Testing practices
    - Security awareness
- Builds a **Developer Skill Graph**

Then:

- Generates personalized interview questions

- Adapts difficulty dynamically

◇ B. Live AI Interview System

Includes:

1 Smart Code Editor

- Monitored typing behavior
- Copy-paste detection
- AI-style pattern detection (LLM fingerprinting heuristics)
- Suspicious latency analysis

Flags:

- Possible AI-generated answers
- Possible plagiarism
- External assistance pattern

---

2 Voice & Meeting Monitoring

Using:

- Whisper (speech-to-text)
- Voice analysis tools
- Behavioral signals

Detect:

- Long unnatural pauses
- Voice switching
- Reading from external source
- Eye direction analysis (if webcam allowed)
- Unusual keystroke rhythm

⚠ Important: This part enters serious legal territory.

◇ C. AI Code Auditing & Runtime Patch Agent

This is the most ambitious part.

Flow:

1. Developer pushes commit.
2. AI Agent:
    - Reviews code.

    - Detects vulnerabilities.
    - Identifies performance issues.

3. If severe issue:

    - Auto-patches the **running instance**.
    - Logs modification.
    - Notifies developer.
    - Requires human validation next commit.

This is essentially:

> AI Runtime Guardian Agent

This must be implemented carefully (see Risks section).

◇ D. Enterprise HR Intelligence Dashboard

Enterprises can:

- Select AI strictness level
- View cheating probability index
- View candidate technical radar chart
- Access historical interviews
- Compare candidates
- Access AI explanations

◇ E. Internal Enterprise Mode

Normal users inside company:

- Access past meetings
- Navigate org data
- Search interview history
- Knowledge base access

---

3 MCP Multi-Agent Architecture

You will use MCP Server as orchestration layer.

🎯 Core Agents

1 Repository Analysis Agent

- Parses Git repos
- Extracts metadata

- Generates skill graph nodes (Neo4j)

2 Interview Generation Agent

- Generates adaptive questions
- Context aware

3 Live Monitoring Agent

- Code typing behavior monitor
- AI suspicion scoring

4 Voice & Behavior Agent

- Speech transcription
- Behavioral anomaly detection

5 Runtime Patch Agent (Critical)

- Security vulnerability detection
- Hot patching
- Rollback management

6 Audit Transparency Agent

- Logs all AI modifications
- Ensures explainability

7 Risk & Bias Evaluation Agent

- Detects bias in evaluation

---

4 Technical Architecture

◇ Backend Core

- **NestJS** → API Gateway
- Python → AI services (LangGraph / LangChain)
- Rust (RIG Agent) → High-performance runtime patch module
- Microservices architecture
- Load balancer

◇ AI Framework

You mentioned:

- LangChain

- LangGraph
- Google ADK
- AGNO

Additional suggestions:

- CrewAI (multi-agent orchestration)
- Haystack (retrieval pipelines)
- LlamaIndex (graph-based retrieval)

LangGraph is strong for:

- Agent state management
- Deterministic workflows

◇ Data Layer

You proposed Neo4j.

That's actually a very smart choice.

Use Neo4j for:

- Developer skill graph
- Interview knowledge graph
- Enterprise org structure

But you still need:

- PostgreSQL → transactional data
- Redis → real-time interview state
- Object storage → logs & recordings

Neo4j should NOT be used alone.

◇ LLM API Usage

You mentioned:

- Whisper
- Grok
- ElevenLabs

Add:

- GPT-4.x or Claude for reasoning
- Code-dedicated models (e.g., Code LLM)
- Guardrails layer (LLM output validation)

---

5 AI Cheating Detection — Critical Analysis

This is extremely difficult.

You CANNOT reliably detect:

- AI-generated code with certainty.
- Someone reading from another screen.

You can:

- Detect anomalies.
- Generate probabilistic scoring.

Never use binary verdict.
Use:

> Risk Probability Index (0-100%)

Otherwise you risk legal problems.

---

6 Runtime AI Auto-Patching — Major Risk

This is dangerous.

Downside:

- AI may introduce new bugs.
- Legal liability if system breaks.
- Security implications.
- Trust issues with developers.

Better alternative:

Instead of auto-patching production:

Implement:

- Shadow patch layer
- Temporary override
- Feature flag system
- Automatic rollback
- Mandatory developer validation

Safer model:
AI suggests patch → Auto-applies in staging → Human validation required for production.

---

7 Business Model

Enterprise Plans:

Plan	AI Strictness	Features
Basic	Low	Repo analysis + interview gen
Pro	Medium	Code editor monitoring
Enterprise	High	Voice + cheating detection
Elite	Advanced	Runtime Guardian + org analytics

8 Ethical & Legal Considerations

You MUST consider:

- GDPR compliance
- Candidate consent
- AI transparency
- Bias mitigation
- Right to explanation
- Data deletion policies

Monitoring voice + gestures can be legally restricted in many countries.

You need:

- Consent system
- Privacy-first architecture

9 Things You Didn’t Mention (But Should)

◇ 1. Explainability Layer

Every AI decision must be explainable.

Example:
"Candidate received 72/100 because:"

- Low test coverage in repos
- Poor commit structure
- Failed concurrency question

Without this:
Your system will be rejected by enterprises.

◇ 2. Bias Detection

AI hiring systems can be biased.

You need:

- Bias monitoring agent
- Fairness scoring

◇ 3. Adversarial Testing

Candidates will try to bypass the system.

You need:

- Red team testing
- Stress testing
- Synthetic cheating simulation

◇ 4. Cost Control System

LLM APIs are expensive.

You need:

- Token usage tracking
- Model routing (cheap vs advanced models)
- Caching layer

🔟 Realistic Downsides

Be honest:

1. This project is HUGE.
2. Cheating detection is unreliable.
3. Runtime patching is dangerous.
4. Legal complexity is high.
5. AI cost can explode.

If done badly:
It becomes a surveillance tool.

If done correctly:
It becomes a serious enterprise SaaS.

🎯 My Professional Advice

For V1:

Focus on:

- Repository Analysis Agent
- Interview Generation Agent
- Skill Graph (Neo4j)
- Live Code Editor with anomaly scoring (basic)
- Enterprise dashboard

Avoid for V1:

- Runtime auto-patching in production
- Aggressive surveillance
- Real-time behavioral analysis

Build modular so you can add later.

🚀 Final Evaluation

This is:

- Architecturally advanced
- AI heavy
- Ethically sensitive
- Enterprise-grade
- Extremely impressive if done correctly

It can become:

> A serious startup-level platform.